

USB CDC 虚拟串口通讯协议 (云台↔miniPC)

云台 MCU (STM32F407) 通过 USB CDC 与 miniPC 进行两路数据交互。

物理接口: USB FS (CDC ACM)

字节序: 多字节字段默认 **小端序 (Little-Endian)**

帧同步

CDC 是连续字节流。miniPC 端按首字节分流：

首字节	类型	TX 帧长	RX 帧长
0x39	视觉	26 字节	19 字节
0xA5	导航	31 字节	15 字节

一、云台 → miniPC

1.1 视觉帧 (26 字节) — 0x39 0x39

偏移	类型	字段	说明
0	uint8	head_one	0x39
1	uint8	head_two	0x39
2	float32	quat_w	IMU 四元数 w (HI12)
6	float32	quat_x	IMU 四元数 x
10	float32	quat_y	IMU 四元数 y
14	float32	quat_z	IMU 四元数 z
18	uint8	bullet_rate	射速 (定值 26)
19	uint8	enemy_color	敌方颜色 (定值 0x52)
20	uint8	vision_mode	视觉模式
21	uint8	tail	帧尾 0xFF
22	uint32	time	自增时间戳 (ms), 大端序

1.2 导航帧 (31 字节) — 0xA5

偏移	类型	字段	说明
0	uint8	head	0xA5
1	float32	x	里程计 x (m)

偏移	类型	字段	说明
5	float32	y	里程计 y (m)
9	float32	yaw	偏航角 (rad)
13	float32	vx	底盘系 x 速度 (m/s)
17	float32	vy	底盘系 y 速度 (m/s)
21	float32	wz	角速度 (rad/s)
25	uint32	time	HAL_GetTick() (ms)
29	uint16	checksum	前 29 字节累加和低 16 位

数据来源：底盘 CAN 上行 (0x310)。调用侧通过 `navigation.setOdom()` / `navigation.setVel()` 填入。

二、miniPC → 云台

2.1 视觉帧 (19 字节) — 0x39 0x39

偏移	类型	字段	说明
0	uint8	head_one	0x39
1	uint8	head_two	0x39
2	int32	pitch_angle	目标 Pitch, $\text{deg} \times 100$, 大端序
6	int32	yaw_angle	目标 Yaw, $\text{deg} \times 100$, 大端序
10	uint8	vision_ready	视觉就绪标志
11	uint8	fire	发射指令
12	uint8	tail	帧尾
13	uint32	time	时间戳 (ms), 大端序
17	uint8	aim_x	瞄准点 x
18	uint8	aim_y	瞄准点 y

pitch/yaw 为 int32 大端序，miniPC 端 $\text{angle_deg} \times 100$ 取整发送。云台 `/100.0` 还原。无 checksum。

2.2 导航帧 (15 字节) — 0xA5

偏移	类型	字段	说明
0	uint8	head	0xA5
1	float32	vx	目标 x 速度 (m/s)

偏移	类型	字段	说明
5	float32	vy	目标 y 速度 (m/s)
9	float32	wz	目标角速度 (rad/s)
13	uint16	checksum	前 13 字节累加和低 16 位

checksum 校验失败则丢弃该帧。

API: `navigation.getTargetVx()`, `navigation.getTargetVy()`, `navigation.getTargetWz()`

三、miniPC 端参考实现

```
def checksum(data):
    return sum(data) & 0xFFFF

# —— 解析 (miniPC 接收) ——

def parse_vision_tx(buf): # 26 字节
    """云台→miniPC 视觉帧"""
    q = struct.unpack_from('<ffff', buf, 2)
    t = struct.unpack_from('>I', buf, 22)[0]
    return q, t

def build_vision_rx(pitch_deg, yaw_deg, ready, fire): # 19 字节
    """miniPC→云台 视觉帧"""
    buf = bytearray(19)
    buf[0:2] = b'\x39\x39'
    struct.pack_into('>ii', buf, 2, int(pitch_deg * 100), int(yaw_deg * 100))
    buf[10] = ready
    buf[11] = fire
    return buf

def parse_nav_tx(buf): # 31 字节, 返回 dict 或 None
    """云台→miniPC 导航帧"""
    if checksum(buf[:29]) != struct.unpack_from('<H', buf, 29)[0]:
        return None
    return struct.unpack_from('<ffffffI', buf, 1)

def build_nav_rx(vx, vy, wz): # 15 字节
    """miniPC→云台 导航帧"""
    buf = bytearray(15)
    buf[0] = 0xA5
    struct.pack_into('<fff', buf, 1, vx, vy, wz)
    struct.pack_into('<H', buf, 13, checksum(buf[:13]))
    return buf

# —— 接收主循环 ——

def recv_loop():
    while True:
        h = read(1)[0]
```

```
if h == 0x39:
    rest = read(25)          # 视觉：收满 26 字节
    # parse_vision_tx(rest) ...
elif h == 0xA5:
    rest = read(30)          # 导航：收满 31 字节
    parsed = parse_nav_tx(b'\xA5' + rest)
else:
    continue # 不同步字节，跳过
```

四、miniPC 只跑导航时

没有视觉代码不影响导航解析。只需在 `recv_loop` 中：

1. 找 `0xA5`
2. 读后续 30 字节
3. checksum 校验 — 通过则解析，失败则回到步骤 1

视觉帧 `0x39` 会被步骤 1 当作不同步字节跳过，不会污染导航数据流。